Image Classification with Vision Transformers: An Experimental Study

Tian Liu
Johns Hopkins University
tliu130@jh.edu

Abstract

Vision transformers have low inductive bias, where the architecture makes very few built-in assumptions about the structure of the data. They lack the translational equivariance of convolutions, so they usually need large datasets to train well. I compared three models of similar size trained from scratch on a small dataset, CIFAR-100, at its native 32×32 resolution: a plain ViT with 4×4 patch embeddings (4.80M parameters), a hybrid that replaces the patch embedding with a ResNet-56 tokenizer (4.60M), and a scaled-down Swin Transformer with shifted window attention (4.25M). All three used the same data splits, augmentation, optimizer, schedule, and 200-epoch budget. The hybrid reached 76.58% test accuracy and a macro F1 of 0.764, compared to 71.52% accuracy for the plain ViT and 70.74% for the Swin. McNemar tests showed the hybrid was significantly better than both other models, while the difference between the plain ViT and the Swin was not significant after a Bonferroni correction. The hybrid's largest gains were on reptiles and invertebrates, while all three models confused the five people classes with each other about equally often. These results suggest that at this data scale, the convolutional bias of the hybrid matters more than windowed attention or a hierarchical design alone.

1 Introduction

Vision Transformers (ViTs) split an image into patches and process them with the same transformer used for text [1]. Unlike convolutional networks, they do not assume that nearby pixels are related or that the same feature can appear anywhere in the image. Every relationship between patches has to be learned from data. This makes ViTs flexible, but it also means they tend to need much more training data than CNNs to reach the same accuracy.

CIFAR-100 is a hard setting for vision transformers. There are only 500 training images per class, the images are only 32×32, and many classes look alike at that size. In this report, I train models from scratch on CIFAR-100 to see how much built-in image structure helps a transformer when data is limited. My primary model is a ResNet-Transformer hybrid, which uses a custom-built ResNet-56-style stem to turn the image into tokens before the transformer. I compare it to a plain ViT baseline with a similar number of parameters. As a third experiment, I also train a scaled-down Swin Transformer, which adds locality and a hierarchy of feature maps through windowed attention, but without convolutions.

To make the comparison fair, all three models have between 4.2M and 4.8M parameters and use the same data splits, augmentation, optimizer, learning-rate schedule, and training budget.

Preprint.

2 Dataset and Preprocessing

2.1 CIFAR-100 Dataset

The CIFAR-100 dataset contains 60,000 images total. It has 50,000 training images and 10,000 test images at 32×32 resolution across 100 classes. There are 600 images per class.

2.2 Preprocessing

I split the 50,000-image CIFAR-100 training set into 45,000 training and 5,000 validation images, using a stratified split so each class has 450 training and 50 validation images. By comparing the MD5 hash of each image's raw pixels, I found 14 pairs of exact duplicates in the training set. I did not remove them, but I made sure no pair was split between training and validation. If the split put one copy in validation, I moved it to training and replaced it with a training image of the same class, so the split sizes and per-class counts did not change. All 14 pairs ended up in the training set, so it has 44,986 unique images. Automated tests (`tests/test_data.py`) verify that the three splits share no indices, that no image appears in both training and validation or in both validation and test, and that the training split shares exactly the 10 exact-duplicate images that the official training and test files already have in common.

2.2.1 Input Resolution

I kept the native CIFAR-100 resolution of 32×32 and did not resize the images. Upscaling to 224×224 adds no new information, and it would make training much more expensive on Colab. With 4×4 patches, a 224×224 image gives 3,136 tokens instead of 64, and attention memory grows quadratically with the number of tokens. At 32×32, all three models fit the resolution evenly. The plain ViT's 4×4 patches and the hybrid's ResNet both give an 8×8 grid of 64 tokens. For the Swin, 2×2 patches give a 16×16 grid, and the 4×4 window divides the 16×16, 8×8, and 4×4 grids of all three stages.

2.2.2 Normalization

I normalized each RGB channel with the mean and standard deviation of the 45,000 training images:

$$\text{mean} = (0.5069,\ 0.4864,\ 0.4409), \quad \text{std} = (0.2674,\ 0.2565,\ 0.2762)$$

I computed these from the training split only, so no validation or test images affect training. The same values are used for the validation and test sets.

2.2.3 Augmentation

Transformers have weaker built-in assumptions about images than CNNs, and 45,000 images is a small training set for them, so I used strong augmentation as regularization. Each training image goes through:

- a random 32×32 crop after 4 pixels of reflection padding,
- a horizontal flip with probability 0.5,
- RandAugment with the torchvision defaults (2 operations, magnitude 9),
- random erasing with probability 0.25.

After batching, each batch is mixed with either MixUp ($\alpha = 0.8$) or CutMix ($\alpha = 1.0$), chosen at random. Both combine pairs of images and mix their labels in the same proportion.

2.2.4 Evaluation Transforms

Validation and test images are only converted to tensors and normalized, with no random augmentation. Random augmentation would make the measured accuracy change from run to run even with the same model, so it would no longer reflect the model's performance on the real images. I also select the best checkpoint by validation loss, which only works if the validation set is the same every epoch. The test set should also match the images the model would see in practice, not distorted versions of them.

3 Model Architectures

3.1 ViT

I implemented the plain ViT as described in Dosovitskiy et al. [1]. I split each 32×32 image into 4×4 patches, giving 64 tokens, and project each patch to $D = 256$ with a single linear layer, with no convolutions. I add a learned position embedding and pass the tokens through 9 pre-norm transformer blocks with 4 heads and an MLP ratio of 2. The main difference from the original ViT is that I use mean pooling over the output tokens instead of a class token. The original paper found the two perform about equally well when the learning rate is tuned, and later work such as Beyer et al. [2] uses mean pooling for smaller ViTs.

3.2 Hybrid

My primary model is a ResNet-Transformer hybrid, following the hybrid architecture in Dosovitskiy et al. [1]. Instead of linear patches, the tokens come from a CIFAR-style ResNet-56 [3]: a 3×3 convolution, then three stages of 9 residual blocks with 16, 32, and 64 channels, where the last two stages halve the resolution. This turns the 32×32 image into an 8×8 feature map, and a 1×1 convolution projects each of the 64 positions to a 256-dimensional token. After that, the model is the same as my plain ViT: a learned position embedding, transformer blocks, and mean pooling. I used 7 blocks instead of 9 so that the two models have parameter counts within 10% of each other.

3.3 Swin

I also implemented a Swin Transformer [4] as a third comparison. Swin restricts attention to small windows and builds a hierarchy of feature maps, like a CNN. Since the original Swin was designed for 224×224 images, I scaled it down for CIFAR: 2×2 patches give a 16×16 grid of 64-dimensional tokens, and I use three stages instead of four, with $(2, 4, 4)$ blocks, $(2, 4, 8)$ heads, and 4×4 windows instead of 7×7. Blocks alternate between regular and shifted windows, using a cyclic shift with a mask so tokens that wrap around the image edge cannot attend to each other. Instead of an absolute position embedding, each block learns a relative position bias. Between stages, patch merging halves the grid and doubles the channels ($16 \times 16 \times 64 \to 8 \times 8 \times 128 \to 4 \times 4 \times 256$). Like my other models, it uses mean pooling before the classifier. Table 1 summarizes the three architectures.

Table 1: Architectures of the three models.

	Plain ViT	Hybrid	Swin
Tokenizer	4×4 patch embedding	ResNet-56 + 1×1 conv	2×2 patch embedding
Tokens	64	64	$256 \to 64 \to 16$
Embedding dim D	256	256	$64 \to 128 \to 256$
Transformer blocks	9	7	$2 + 4 + 4 = 10$
Heads (head dim)	4 (64)	4 (64)	2, 4, 8 (32)
MLP ratio r	2	2	4
Attention	global	global	windowed 4×4, shifted
Downsampling	none	stride-2 convs ($32 \to 8$)	patch merging ($16 \to 4$)
Position encoding	learned absolute	learned absolute	relative bias
Pooling	mean	mean	mean
Stochastic depth	$0 \to 0.1$	$0 \to 0.1$	$0 \to 0.1$
Trainable parameters	4,799,076	4,604,084	4,247,064

4 Parameter and Compute Analysis

4.1 Parameters in One Transformer Block

For embedding dimension D and MLP ratio r, the attention layer has a $D \to 3D$ query-key-value projection and a $D \to D$ output projection, the MLP has $D \to rD$ and $rD \to D$ layers, and the two

LayerNorms each have $2D$ parameters:

$$
\begin{aligned}
P_{\text{attn}} &= 4D^2 + 4D \\
P_{\text{MLP}} &= 2rD^2 + (r+1)D \\
P_{\text{norm}} &= 4D \\
P_{\text{block}} &= (4+2r)D^2 + (r+9)D
\end{aligned}
$$

With $D = 256$ and $r = 2$:

$$
\begin{aligned}
P_{\text{attn}} &= 4(256)^2 + 4(256) = 263{,}168 \\
P_{\text{MLP}} &= 2(2)(256)^2 + (2+1)(256) = 262{,}912 \\
P_{\text{norm}} &= 4(256) = 1{,}024 \\
P_{\text{block}} &= (4+2(2))(256)^2 + (2+9)(256) = 527{,}104
\end{aligned}
$$

All transformer blocks in both plain ViT and ResNet-ViT hybrids have only one distinct block type with $D = 256$ and $r = 2$. Swin has 3 distinct types, one per stage, because D doubles every stage, and each block has an extra relative position bias table. A Swin block has everything a ViT block has in addition to the relative position bias table with $(2w-1)^2$ offsets for every head. Dimensionality and number of heads double every stage.

$$
P_{\text{block,Swin}} = (4+2r)D^2 + (r+9)D + (2w-1)^2 h
$$

With $(D, h) = (64, 2), (128, 4), (256, 8)$, $r = 4$, $w = 4$:

$$
\begin{aligned}
P_{\text{block, stage1}} &= (4+2(4))(64)^2 + (4+9)(64) + (2(4)-1)^2(2) = 50{,}082 \\
P_{\text{block, stage2}} &= (4+2(4))(128)^2 + (4+9)(128) + (2(4)-1)^2(4) = 198{,}468 \\
P_{\text{block, stage3}} &= (4+2(4))(256)^2 + (4+9)(256) + (2(4)-1)^2(8) = 790{,}152
\end{aligned}
$$

4.1.1 Why does the parameter count grow approximately quadratically with D?

Every weight matrix in a block maps D-dimensional input to an output whose size is proportional to D. For the attention layer, the qkv projection is $D \times 3D$ and the linear output projection is $D \times D$. The two MLP layers are $D \times rD$ and $rD \times D$ respectively. Therefore, the attention and MLP layers have $(4+2r)D^2$ weights, so they grow quadratically. Only the biases and LayerNorm parameters grow linearly, but they are a small percentage ($\approx 0.5\%$ with $D = 256$ and $r = 2$) of the overall parameters.

4.1.2 Why does changing the number of attention heads not necessarily change the parameter count when D remains fixed?

Every attention weight matrix only depends on D and not on the number of heads. Increasing the number of heads means splitting the dimensionality for each head (e.g. $D = 256$, $h = 4$ would result in 4 heads of 64 dimensions). The MLP and LayerNorms act on the full D-dimensional vector and do not involve or depend on the number of heads.

For Swin models, each relative position bias table is $(2w-1)^2 h$, so increasing h does result in more parameters per block (49 per head in my model). In my Swin model, at the third stage with 8 heads, 392 of the 790,152 parameters come from the bias table, which is negligible (0.05%).

4.1.3 Why increasing model width may increase parameters more quickly than increasing model depth?

Adding depth to the model adds the same P_{block}, so the total grows by a constant amount per block. The growth is linear when adding depth, while increasing the width D grows the parameter count quadratically. For my ViT ($D = 256$, $r = 2$, 9 blocks), doubling the depth to 18 blocks doubles the encoder from 4.74M to 9.49M parameters, while doubling the width to $D = 512$ nearly quadruples it to 18.9M. However, for small increases, widening does not always add more parameters than deepening. For example, increasing D from 256 to 272 adds around 68k parameters per block, and 610k total for 9 blocks, which is about the same as adding one block.

4.2 Total Model Parameters

4.2.1 Plain ViT

With $C = 3$ input channels, patch size $p = 4$, $N = 64$ tokens, $L = 9$ blocks, $D = 256$, and $K = 100$ classes:

$$
\begin{aligned}
P_{\text{patch}} &= (Cp^2 + 1)D = (3(4)^2 + 1)256 = 12{,}544 \\
P_{\text{class}} &= 0 \quad \text{(mean pooling; a class token would add } D\text{)} \\
P_{\text{pos}} &= ND = 64(256) = 16{,}384 \\
P_{\text{blocks}} &= L \cdot P_{\text{block}} = 9(527{,}104) = 4{,}743{,}936 \\
P_{\text{norm}} &= 2D = 2(256) = 512 \\
P_{\text{head}} &= DK + K = 256(100) + 100 = 25{,}700 \\
P_{\text{total}} &= 4{,}799{,}076
\end{aligned}
$$

Table 2: Hand-calculated and PyTorch-reported parameter counts for the plain ViT.

Hand-calculated parameter count	4,799,076
PyTorch-reported parameter count	4,799,076
Absolute difference	0
Percentage difference	0.0%

Table 2 compares the two counts. There was no discrepancy between the number of parameters I calculated and the number PyTorch reported.

4.2.2 ResNet-Transformer Hybrid

With $C = 3$ input channels, $n = 9$ blocks per stage, stage widths $(c_1, c_2, c_3) = (16, 32, 64)$, and $\delta = 1$ when a stage changes channels (0 otherwise):

$$
\begin{aligned}
P_{\text{stem}} &= 9Cc_1 + 2c_1 \\
P_{\text{stage}}(c_{\text{in}}, c_{\text{out}}) &= 9c_{\text{in}}c_{\text{out}} + (18n - 9)c_{\text{out}}^2 + 4nc_{\text{out}} + \delta\,(c_{\text{in}}c_{\text{out}} + 2c_{\text{out}}) \\
P_{\text{backbone}} &= P_{\text{stem}} + P_{\text{stage}}(16, 16) + P_{\text{stage}}(16, 32) + P_{\text{stage}}(32, 64) \\
P_{\text{BN-stats}} &= (1 + 2n)(2c_1 + 1) + (2n + 1)(2c_2 + 1) + (2n + 1)(2c_3 + 1)
\end{aligned}
$$

Substituting:

$$
\begin{aligned}
P_{\text{stem}} &= 9(3)(16) + 2(16) = 464 \\
P_{\text{stage}}(16, 16) &= 9(16)(16) + 153(16)^2 + 36(16) = 42{,}048 \\
P_{\text{stage}}(16, 32) &= 9(16)(32) + 153(32)^2 + 36(32) + (16)(32) + 2(32) = 163{,}008 \\
P_{\text{stage}}(32, 64) &= 9(32)(64) + 153(64)^2 + 36(64) + (32)(64) + 2(64) = 649{,}600 \\
P_{\text{backbone}} &= 464 + 42{,}048 + 163{,}008 + 649{,}600 = 855{,}120 \\
P_{\text{BN-stats}} &= 19(33) + 19(65) + 19(129) = 4{,}313
\end{aligned}
$$

With $D = 256$, $N = 64$ tokens, $L = 7$ blocks, and $K = 100$ classes:

$$
\begin{aligned}
P_{\text{feature}} &= c_3 D + D = 64(256) + 256 = 16{,}640 \\
P_{\text{pos}} &= ND = 64(256) = 16{,}384 \\
P_{\text{blocks}} &= L \cdot P_{\text{block}} = 7(527{,}104) = 3{,}689{,}728 \\
P_{\text{norm}} &= 2D = 2(256) = 512 \\
P_{\text{head}} &= DK + K = 256(100) + 100 = 25{,}700 \\
P_{\text{total}} &= 4{,}608{,}397
\end{aligned}
$$

Table 3: Hand-calculated and PyTorch-reported parameter counts for the hybrid.

Hand-calculated parameter count	4,608,397
PyTorch-reported parameter count	4,604,084
Absolute difference	4,313
Percentage difference	0.094%

The difference in parameters in Table 3 comes entirely from the batch normalization layers in the ResNet. Each of the 57 BatchNorm layers (1 for stem, 18, 19, and 19 for stages 1, 2, and 3 respectively) stores, in addition to its learned scale and shift ($2c$ parameters, included in both counts), a running mean and variance ($2c$ values) and a counter of batches seen (1 value). These values are updated during training as moving averages of each batch's statistics rather than by gradient descent, so they are registered as buffers, which PyTorch excludes from `model.parameters()`. My hand calculation counted every stored value, including the 4,313 buffer values.

4.2.3 Swin

With $C = 3$ input channels, patch size $p = 2$, MLP ratio $r = 4$, window size $w = 4$, $K = 100$ classes, $(2, 4, 4)$ blocks, widths $(D_1, D_2, D_3) = (64, 128, 256)$, and heads $(h_1, h_2, h_3) = (2, 4, 8)$ in stages 1–3, and token-grid sides $(G_1, G_2) = (16, 8)$ in stages 1 and 2:

$$
\begin{aligned}
P_{\text{patch}} &= (Cp^2 + 1)D_1 = (3(2)^2 + 1)(64) = 832 \\
P_{\text{blocks}} &= 2(12D_1^2 + 9D_1) + 4(12D_2^2 + 9D_2) + 4(12D_3^2 + 9D_3) \\
&= 2(12(64)^2 + 9(64)) + 4(12(128)^2 + 9(128)) + 4(12(256)^2 + 9(256)) \\
&= 99{,}456 + 791{,}040 + 3{,}154{,}944 = 4{,}045{,}440 \\
P_{\text{merge}} &= 8D_1^2 + 8D_2^2 = 8(64)^2 + 8(128)^2 = 163{,}840 \\
P_{\text{pos}} &= 2(2w-1)^2 h_1 + 4(2w-1)^2 h_2 + 4(2w-1)^2 h_3 + 10w^4 + 1\left(\tfrac{G_1}{w}\right)^2 w^4 + 2\left(\tfrac{G_2}{w}\right)^2 w^4 \\
&= 2(7)^2(2) + 4(7)^2(4) + 4(7)^2(8) + 10(4)^4 + 1(4)^2(4)^4 + 2(2)^2(4)^4 \\
&= 2{,}548 + 2{,}560 + 6{,}144 = 11{,}252 \\
P_{\text{norm}} &= 2(4D_1) + 4(4D_2) + 4(4D_3) + 8D_1 + 8D_2 + 2D_3 \\
&= 2(4)(64) + 4(4)(128) + 4(4)(256) + 8(64) + 8(128) + 2(256) = 8{,}704 \\
P_{\text{head}} &= D_3 K + K = 256(100) + 100 = 25{,}700 \\
P_{\text{total}} &= 4{,}255{,}768
\end{aligned}
$$

Table 4: Hand-calculated and PyTorch-reported parameter counts for the Swin.

Hand-calculated parameter count	4,255,768
PyTorch-reported parameter count	4,247,064
Absolute difference	8,704
Percentage difference	0.205%

Buffers also explain the discrepancy in Table 4 between my hand-calculated values and the PyTorch-reported values for Swin. The reason is different in this case because Swin's LayerNorm does not keep running statistics, so all its parameters are learned and trained. The difference instead comes from two fixed structures included in P_{pos}: the relative-position index maps ($2{,}560$ values) and the shift masks ($6{,}144$ values). Both are determined entirely by window and grid size and are never updated, so they are registered as buffers and excluded from `model.parameters()`. The two fixed tables account for the $8{,}704$ value difference.

Both comparison models are within 10% of the primary hybrid's $4{,}604{,}084$ trainable parameters: the plain ViT ($4{,}799{,}076$) is 4.2% larger and the Swin ($4{,}247{,}064$) is 7.8% smaller.

4.3 Training Data Relative to Model Size

I found 14 exact duplicate pairs in the training data, so there were only 44,986 unique training images. I will be using that as $n_{train} = 44{,}986$ for calculating κ and ρ for my models (Table 5).

Table 5: Parameters per unique training image (κ) and unique training images per million parameters (ρ), with $n_{\text{train}} = 44{,}986$.

	Plain ViT	Hybrid	Swin
κ	106.68	102.34	94.41
ρ	9373.9	9770.9	10592.3

I will be using $n_{train} = 45{,}000$ for Model A and Model B calculations (Table 6).

Table 6: κ and ρ for the two hypothetical models, with $n_{\text{train}} = 45{,}000$.

	Model A	Model B
Parameters	24M	86M
κ	533.3	1911.1
ρ	1875.0	523.3

The multiplicative difference between the two models is (86M/24M) which is ≈ 3.583.

The larger model may require stronger augmentation or regularization because there are fewer training images per parameter, which can lead to overfitting. More parameters give the model more freedom to fit the training set exactly and to memorize individual images. Augmentation would enlarge the dataset, increasing the ratio of training images to parameters, and regularization can help restrict how freely the parameters can fit the training set.

These ratios do not prove that the small model will generalize better because κ treats every parameter equally, but that is not the case in practice. The model architecture is what determines what each parameter represents, so two models with the same number of parameters can behave very differently.

Inductive biases for locality, hierarchy, and parameter sharing, like in CNNs, help a model learn useful features from fewer examples. Because of this, κ and ρ are not a fair comparison across different architectures. In a CNN, the same kernel is used at every position, so each parameter gets trained on much more data than κ suggests. My results show this. The hybrid has all three biases from its ResNet backbone and generalized the best, even though its ρ was in the middle. Swin has locality and hierarchy from its windows but no convolutions, and it had the highest ρ but the lowest test accuracy.

4.4 Training Exposure

E was 200 epochs for plain ViT and Swin, while it was 195 epochs for the ResNet-Transformer hybrid because of early stopping. I use $n_{\text{train}} = 44{,}986$ and the parameter counts from PyTorch for each model (Table 7).

Table 7: Total image presentations during training and presentations per million parameters.

	Plain ViT	Hybrid	Swin
$N_{\text{presentations}}$	8,997,200	8,772,270	8,997,200
ρ_E	1,874,778	1,905,324	2,118,452

Training for 50 epochs does not create 50 times as many independent training images, because every epoch shows the model the same underlying training images.

Stochastic augmentation changes the repeated presentations by giving a random new version of each image, transforming it (e.g., with RandAugment or random horizontal flips) to prevent pixel-level memorization.

Augmented images are not equivalent to newly collected labeled data, because an augmented image still shows the same object and only adds the invariances that were chosen. New data is required to bring variation that augmentation cannot create.

Both models should receive the same training budget; otherwise, the accuracy gap would confound architecture with compute.

4.5 Global and Local Attention Cost

My ViT uses global attention, so each attention layer produces an attention-score tensor of $A_{\text{global}} = BhS^2$ elements. With a batch size of $B = 128$, $h = 4$ heads, and a sequence length of $S = 64$, the count is $128(4)(64)^2 = 2{,}097{,}152$ per attention layer. The attention-score count for my ResNet-Transformer hybrid is also $2{,}097{,}152$. Its ResNet outputs an 8×8 feature map, so the transformer receives the same 64-token sequence as the ViT, with the same 4 heads. My Swin model uses windowed attention ($A_{\text{window}} = BhNM^2$, with $M^2 = 16$ tokens per window), so its attention-score count differs by stage: $128(2)(256)(16) = 1{,}048{,}576$ in stage 1, $128(4)(64)(16) = 524{,}288$ in stage 2, and $128(8)(16)(16) = 262{,}144$ in stage 3, per block.

The memory for one attention-score tensor is $A \times b$, where $b = 4$ bytes per element in FP32 and $b = 2$ bytes in FP16. For the ViT and the hybrid, each layer's tensor requires $2{,}097{,}152 \times 4 = 8{,}388{,}608$ bytes (8.39 MB) in FP32 and 4.19 MB in FP16. Because training keeps every layer's tensor for backpropagation, the totals are 75.50 MB (FP32) and 37.75 MB (FP16) for the ViT's 9 layers, and 58.72 MB and 29.36 MB for the hybrid's 7 layers. For the Swin, each block requires 4.19, 2.10, and 1.05 MB in FP32 (2.10, 1.05, and 0.52 MB in FP16) in stages 1–3, for a total of 20.97 MB in FP32 and 10.49 MB in FP16.

Patch size is what sets the sequence length. Splitting a 32×32 image into $p \times p$ patches gives $S = (32/p)^2$ tokens. Global attention compares each token with every other token, so its cost grows quadratically, by S^2. Halving the patch size would quadruple the sequence length, and thus make attention 16 times more expensive. The cost of windowed attention grows linearly with the number of tokens because each token only attends to the M^2 tokens in its window. A larger window size lets each token see more of the image, but it raises the cost per token.

Both primary designs add locality in different ways. In the Swin, each token attends only to the 16 tokens in its 4×4 window, so attention is local by construction. Every second block shifts the windows by 2 tokens, which lets information pass between neighboring windows. In the hybrid, each 3×3 convolution combines only neighboring pixels, and stacking 55 of them grows the receptive field gradually before the transformer sees any tokens. The two parameter-matched models are not compute-matched: they have different attention memory, and as discussed in Section 8.4, the hybrid's epochs also take longer.

5 Experimental Setup

All models used the same training, validation, and test splits with the same input resolution of 32×32. All data augmentations were the same as described in Section 2.2.3. The optimizer AdamW was used for all three runs with a learning rate of 5×10^{-4} and a weight decay of 0.05. I applied weight decay only to weight matrices, not to biases, normalization layers, position embeddings, or the Swin's relative position bias tables. The learning rate warmed up linearly over the first 10 epochs and then followed a cosine decay to zero, updated every batch (Figure 3).

I trained with a batch size of 128, with label smoothing of 0.1, gradient clipping at a norm of 1.0, and a drop path rate of 0.1. All three models had the same budget of 200 epochs and the same early stopping rule, which stops training after 30 epochs without improvement in validation loss. The hybrid stopped at epoch 195, but every model's best checkpoint came before that (epoch 165 for the hybrid, 183 for the plain ViT, and 178 for the Swin), so the five missing epochs did not affect the results. All runs used bf16 mixed precision on a single NVIDIA L4 GPU and the same random seed (2026). All models were evaluated the same way: the checkpoint with the lowest validation loss

was run once on the 10,000 test images using only the deterministic transforms, and the same script computed every metric. The experimental setup is summarized in Table 8.

To test whether differences in accuracy between two models are significant, I used McNemar's test with continuity correction on their paired test-set predictions, since all three models were evaluated on the same test images. I applied a Bonferroni correction for the three pairwise comparisons, because running several tests increases the chance that one of them appears significant by chance. The results are computed in `stats.ipynb`.

Table 8: Training settings, shared by all three models.

Setting	**Value**
Epochs	200 (early stopping, patience 30)
Batch size	128
Initial learning rate	5×10^{-4} (peak, after warmup)
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Weight decay	0.05 (weight matrices only)
Learning-rate schedule	Linear warmup, then cosine decay to 0
Warmup duration	10 epochs
Dropout	None
Stochastic depth	0.1 (increasing linearly from 0 over the blocks)
Label smoothing	0.1
Gradient clipping	Max norm 1.0
Mixed precision	bf16
Random seed	2026
Hardware	$1 \times$ NVIDIA L4 GPU (Google Colab)

6 Results

6.1 Metrics

Table 9 reports the full set of test metrics. The hybrid is best on every metric, and the ranking of the three models is the same for all of them. Each model predicts every class between 75 and 128 times, compared to 100 true images per class, so no model is heavily biased toward a few classes. Weighted F1 equals macro F1, and macro recall equals top-1 accuracy, because the CIFAR-100 test set has exactly 100 images per class. The ROC-AUC values are all above 0.987 and much closer together than the accuracies, since AUC measures how well each model ranks the correct class against the others rather than whether its top prediction is correct. Per-class precision, recall, and F1 for all 100 classes are too long to include here, so they are saved in `logs/hybrid_seed2026_per_class.csv`, `logs/vit_seed2026_per_class.csv`, and `logs/swin_seed2026_per_class.csv`. Figure 1 shows the row-normalized confusion matrices, and their most common errors are analyzed in Section 7.

Table 9: Test-set results for the three models. ROC-AUC is computed from the softmax class probabilities.

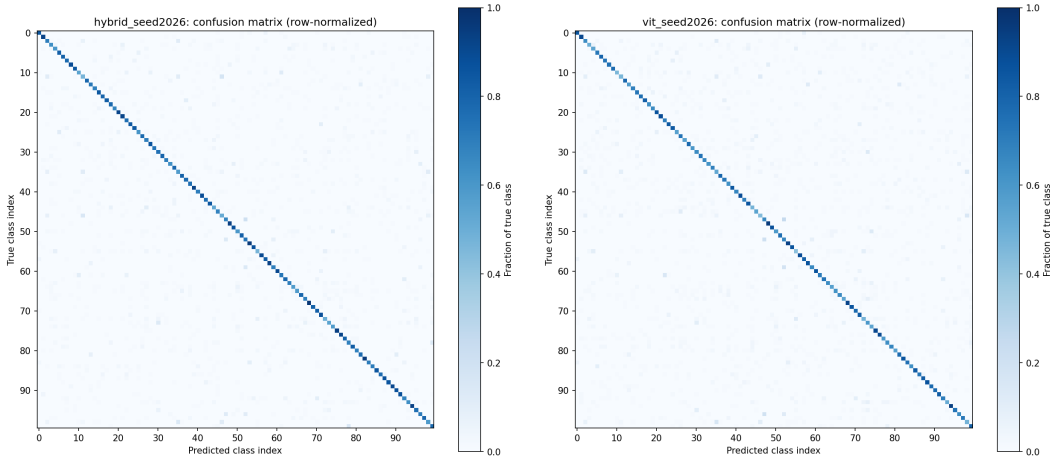
Metric	**Hybrid**	**Plain ViT**	**Swin**
Top-1 accuracy	**0.7658**	0.7152	0.7074
Macro precision	**0.7662**	0.7167	0.7067
Macro recall	**0.7658**	0.7152	0.7074
Macro F1	**0.7643**	0.7135	0.7050
Weighted F1	**0.7643**	0.7135	0.7050
Macro ROC-AUC (OvR)	**0.9920**	0.9876	0.9873

6.2 Comparison

Table 10 compares the three models. The hybrid reached the highest test accuracy at 76.58%, which is 5.06 points higher than the plain ViT and 5.84 points higher than the Swin, even though it has fewer parameters than the plain ViT. The Swin has the fewest parameters per training image but the lowest accuracy, so having more data per parameter did not lead to better results across architectures. For every model, the validation and test accuracy are within 0.6 points of each other, so the validation set gave a reliable estimate of test performance.

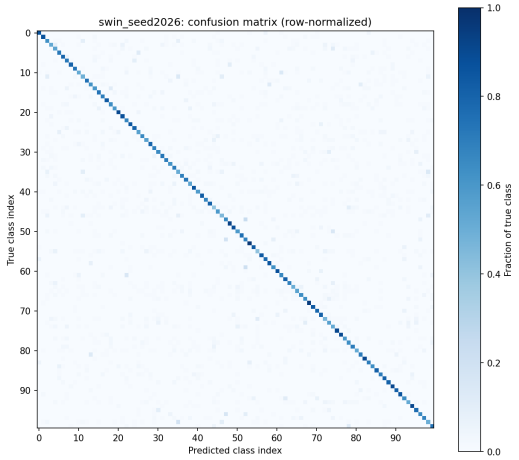
Table 10: Summary of the three models. Parameters per training image is $\kappa = P/n_{\text{train}}$ with $n_{\text{train}} = 44{,}986$. Validation accuracy is at the checkpoint with the lowest validation loss, which is the checkpoint evaluated on the test set. Swin tokens are listed per stage.

Model	Parameters	Params per train image	Tokens	Attention type	Best val accuracy	Test accuracy	Macro F1
Plain ViT	4,799,076	106.68	64	Global	0.7210	0.7152	0.7135
Hybrid	4,604,084	102.34	64	Global	**0.7670**	**0.7658**	**0.7643**
Swin	4,247,064	94.41	256/64/16	Windowed	0.7048	0.7074	0.7050



(a) Hybrid

(b) Plain ViT



(c) Swin

Figure 1: Row-normalized test-set confusion matrices. Classes are indexed in CIFAR-100's alphabetical order (0 = apple, 99 = worm).

6.3 Fitting and Stability

Figure 2 shows the training and validation curves. All three models appear to slightly underfit the training data rather than overfit it. Training accuracy stays around 0.45–0.52 while validation accuracy reaches 0.71–0.77, but this gap does not mean much on its own, because training accuracy is measured on MixUp/CutMix images against their original labels. The clearer signs are the trends. Validation loss never rises again for any model: each model's best checkpoint is in the last 40 epochs, and the final validation loss is within 0.04 of the best. Training loss is also still decreasing at the end, and validation accuracy improved by 1–1.5 points after epoch 150. The curves flatten because the cosine schedule brings the learning rate to zero, not because the models stopped improving. With strong augmentation, label smoothing, and drop path all applied together, the models are probably regularized more than they need to be for 200 epochs, and a longer schedule or weaker augmentation would likely improve all three.

The plain ViT and the Swin behave almost the same: their training curves overlap, and both are still improving slowly at epoch 200. The hybrid fits the training data better than either, with the lowest training and validation loss after 10 epochs. It is the only model that triggered early stopping, at epoch 195, but its validation loss had flattened rather than increased, so this is a plateau, not overfitting.

None of the models were unstable during optimization. There were no loss spikes, divergence, or NaN values, and the warmup and gradient clipping kept early training smooth. The hybrid's validation loss is noisier than the others' (17 epoch-to-epoch jumps above 0.05, compared to 3 for the ViT and 1 for the Swin), but it always recovers the next epoch, and the overall trend is steady.

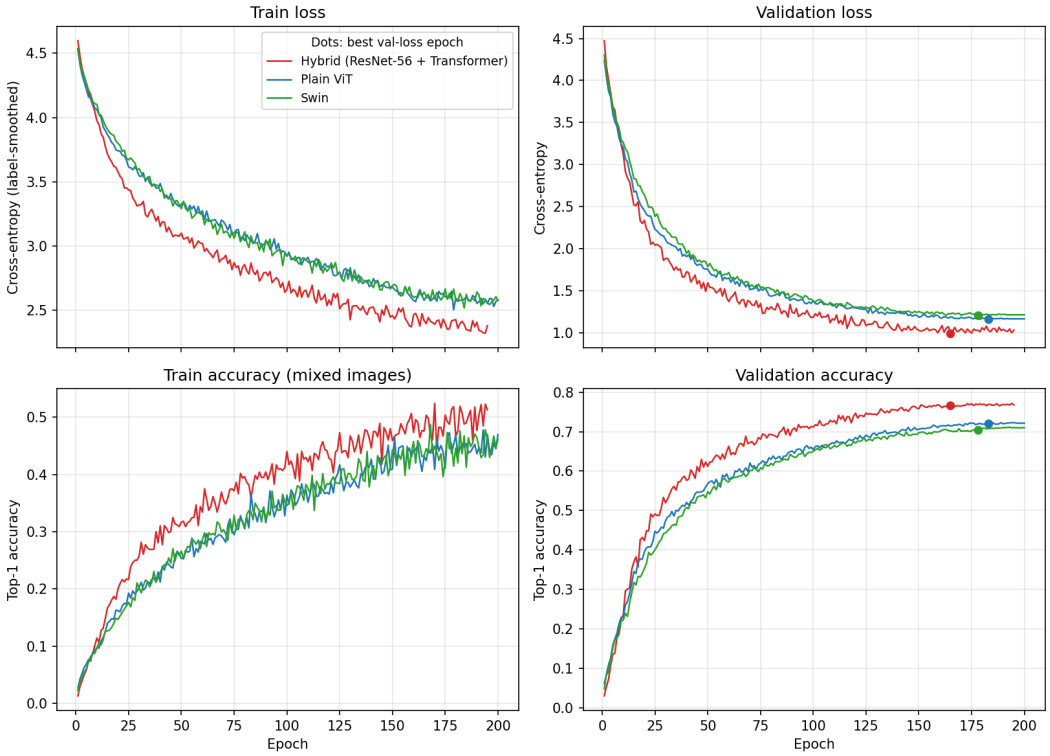


Figure 2: Training and validation loss and accuracy by epoch for all three models. Dots mark each model's lowest validation loss, the checkpoint evaluated on the test set. The hybrid stopped early at epoch 195. Training accuracy is measured on MixUp/CutMix images against their original labels, and training loss includes label smoothing, so both are worse than the validation values.

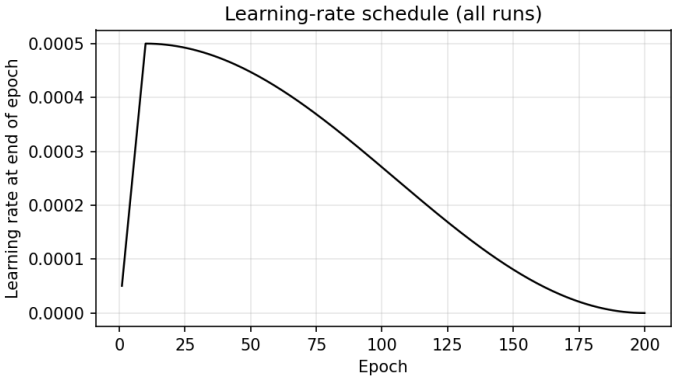


Figure 3: Learning rate by epoch, shared by all three models: linear warmup over 10 epochs to 5×10^{-4}, then cosine decay to 0. The rate is updated every batch; the plot shows its value at the end of each epoch.

7 Error Analysis

Table 11 lists each model's best and worst classes. The best classes are mostly objects with a distinctive shape or color, such as sunflower, lawn mower, motorcycle, and road. Otter, seal, man, and lizard are in the bottom five for all three models.

Table 12 shows three confusion patterns shared by all three models. The first is trees: maple tree $\rightarrow$ oak tree is the single most common error for every model. The second is people: the five people classes are confused with each other about equally often by all three models (170, 170, and 165 errors). The third is small and aquatic mammals, such as mouse $\rightarrow$ shrew and seal $\rightarrow$ otter. In each pattern, the classes look alike at 32×32 and differ mainly in fine detail. Compared to the plain ViT, the hybrid's largest accuracy gains were on the reptile and invertebrate superclasses (10.2 points each). On people it also gained 5.8 points, but by making fewer errors outside the people group, not fewer confusions within it.

Figures 4–6 show each model's most confident errors. Some of the same images fool more than one model: the same lobster image is predicted as a crab by both the ViT and the Swin, the same seal image as an otter by both, and the same rabbit image as a kangaroo by the hybrid and the ViT. A few of these images are ambiguous, such as the "lobster" that looks like a crab, so some confident errors may reflect the image or its label rather than the model.

Table 11: Classes with the highest and lowest test F1 scores for each model.

	Hybrid	Plain ViT	Swin
Highest F1			
1	sunflower (0.941)	lawn mower (0.899)	road (0.900)
2	lawn mower (0.922)	motorcycle (0.885)	chair (0.892)
3	motorcycle (0.922)	road (0.880)	sunflower (0.888)
4	wardrobe (0.916)	wardrobe (0.879)	orange (0.882)
5	tractor (0.910)	skyscraper (0.878)	skunk (0.871)
Lowest F1			
1	boy (0.497)	otter (0.418)	lizard (0.421)
2	seal (0.513)	woman (0.464)	otter (0.453)
3	otter (0.531)	lizard (0.475)	seal (0.468)
4	man (0.531)	seal (0.478)	man (0.479)
5	lizard (0.546)	man (0.486)	shrew (0.484)

Table 12: Common confusion patterns. Each entry is the number of test images of the true class (out of 100) predicted as the other class. The last three rows count all errors where both the true and predicted class are in the same CIFAR-100 superclass.

Pattern	**True → predicted**	**Hybrid**	**Plain ViT**	**Swin**
Trees	maple tree → oak tree	21	25	24
	oak tree → maple tree	13	21	18
	pine tree → oak tree	14	18	21
	willow tree → maple tree	12	19	16
People	woman → girl	14	16	18
	boy → man	17	12	10
	man → boy	15	8	9
Small and aquatic mammals	mouse → shrew	10	13	15
	seal → otter	8	12	11
Errors within trees (5 classes)		106	129	130
Errors within people (5 classes)		170	170	165
Errors within aquatic mammals (5 classes)		61	67	64



Figure 4: The 12 misclassified test images the hybrid was most confident about. Each title gives the true label with the probability the model assigned to it, and the predicted label with its predicted probability.



Figure 5: The 12 most confident misclassifications of the plain ViT.

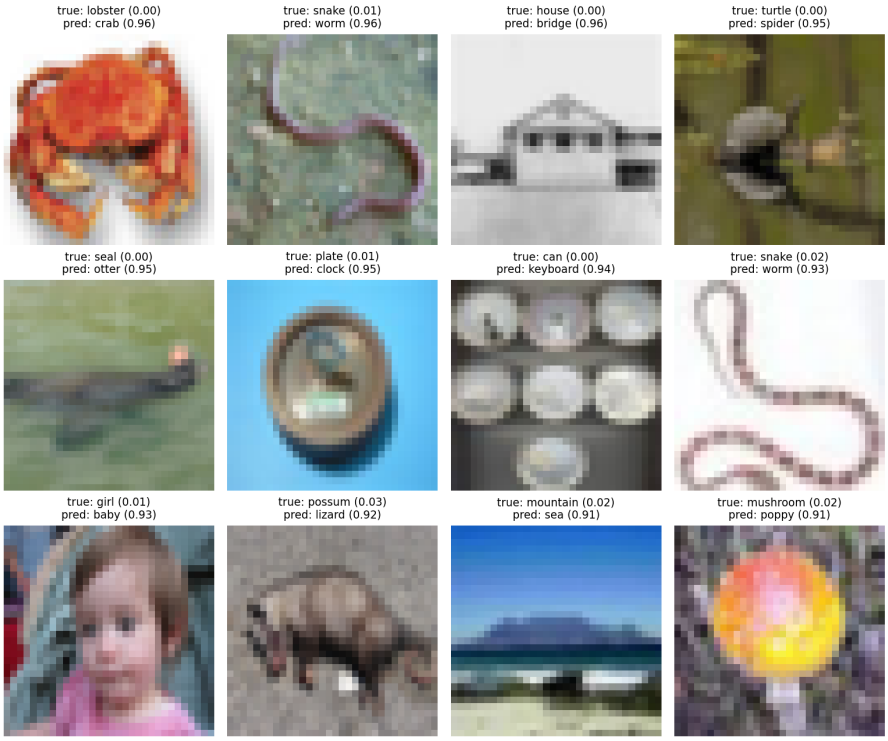


Figure 6: The 12 most confident misclassifications of the Swin.

8 Discussion

8.1 Did the primary model outperform the parameter-matched plain ViT?

Yes. The hybrid reached 76.58% test accuracy compared to 71.52% for the plain ViT, with 4.2% fewer parameters. It was also better on every other metric in Table 9. The difference is statistically significant: the hybrid was correct on 1,127 test images that the ViT got wrong, while the ViT was correct on only 621 that the hybrid got wrong (McNemar $\chi^2 = 145.9$, $p < 10^{-30}$).

8.2 Which model learned more quickly?

Per epoch, the hybrid learned fastest. It reached 50%, 60%, and 70% validation accuracy at epochs 28, 42, and 85, compared to epochs 35, 63, and 134 for the plain ViT and 39, 70, and 160 for the Swin. However, each hybrid epoch took 23.6 seconds on average, compared to 14.4 seconds for the ViT and 17.0 seconds for the Swin. In wall-clock time, the hybrid and the ViT reached 70% at almost the same point (33.6 and 32.1 minutes), while the Swin took 45.2 minutes. So the hybrid learns more from each image, but not faster in real time.

8.3 Which model showed more overfitting?

None of the models clearly overfit. The hybrid showed the most signs of it, but they were small. It was the only model to trigger early stopping, and its final validation loss (1.029) was 0.035 above its best (0.994), compared to 0.007 for the ViT and 0.005 for the Swin. Its validation loss flattened instead of rising, and validation accuracy kept improving slightly, so this looks like a plateau. The hybrid also fit the training data best, with the lowest training loss after 10 epochs, which is expected for the model with the strongest inductive bias.

8.4 Did parameter matching produce similar computational costs?

No. Matching parameters did not match compute. The hybrid's epochs took 64% longer than the ViT's, even though it has fewer parameters, most likely because its ResNet runs convolutions over the full 32×32 image before the transformer. Attention memory also differs: one batch's attention scores take 75.50 MB in FP32 for the ViT, 58.72 MB for the hybrid, and 20.97 MB for the Swin (Section 4.5). Parameter count measures how much a model stores, not how much work it does per image.

8.5 What best explains the differences?

Architectural inductive bias explains the results best. Parameter count does not: the three models are within 12% of each other, and the model with the most parameters (the plain ViT) was in the middle. Sequence length does not either: the plain ViT and the hybrid both use 64 tokens, yet they differ by 5.06 points. Attention structure alone also does not explain it: the Swin's windowed attention gave no significant improvement over the ViT's global attention (70.74% vs. 71.52%, McNemar $p = 0.127$ after Bonferroni correction). The one large difference comes from the convolutional tokenizer. One caveat is that the hybrid differs from the ViT in more than inductive bias, since its ResNet also adds 56 layers of depth, so this experiment cannot fully separate the two.

8.6 Did locality and hierarchical feature processing improve performance?

It depends on how they are built in. Through convolutions in the hybrid, locality and hierarchy improved accuracy by about 5 points. Through windowed attention and patch merging in the Swin, they did not: the Swin was slightly worse than the plain ViT, and the difference was not significant. One possible reason is that convolutions apply a small local filter that is built into the architecture, while windowed attention still has to learn how tokens in a window relate to each other. The Swin was also scaled down heavily from its original design and trained with settings shared with the other models rather than tuned for it.

8.7 What additional experiment would most strengthen the conclusion?

The most useful experiment would be an ablation of the tokenizer. Keeping the same transformer and parameter budget, I would replace the ResNet-56 with tokenizers of increasing convolutional depth, from a plain patch embedding to a small convolutional stem to the full ResNet. This would separate the effect of convolutional bias from the extra depth the ResNet adds. Each model should be trained with at least three seeds, since this report used one seed per model and cannot show how much the results vary between runs.

9 Limitations

Each model was trained with only one random seed, so I cannot tell how much the results would change between runs. This matters most for the plain ViT and the Swin, whose test accuracies differ by only 0.78 points.

The hybrid differs from the plain ViT in more than its inductive bias. Its ResNet-56 tokenizer adds 56 convolutional layers of depth before the transformer, so my experiments cannot separate the effect of convolutional bias from the effect of the extra depth.

All three models used the same hyperparameters. Settings that work well for one architecture may not be the best for another, especially for the Swin, which I scaled down heavily from its original design. In my Swin, the third stage has a 4×4 grid that fits in a single window, so that stage uses global attention rather than windowed attention.

I did not log accuracy on the clean, unaugmented training images, so I cannot directly measure how well each model fits the training data. My conclusions about underfitting rely on the trends in the training and validation curves instead. All models were also still improving slightly when the learning rate reached zero, so a longer schedule could change the gaps between them.

I measured computational cost with time per epoch and attention memory, not FLOPs. The epoch times include data loading and validation, so they are not a pure measure of model compute.

Finally, all results are on CIFAR-100 at 32×32 resolution. The conclusions may not hold for larger images or larger datasets, where transformers are known to depend less on built-in image structure. Ten test images are exact duplicates of training images. I kept them so the test set stays standard, and I did not check for near-duplicates.

10 Conclusion

I trained three transformer models of similar size from scratch on CIFAR-100: a plain ViT, a ResNet-Transformer hybrid, and a scaled-down Swin Transformer. The hybrid was clearly the best, reaching 76.58% test accuracy, which is 5.06 points above the plain ViT and 5.84 points above the Swin, while having fewer parameters than the plain ViT. It also learned faster per epoch and its largest gains were on reptiles and invertebrates. The Swin's windowed attention and hierarchy did not improve on the plain ViT, and the difference between those two models was not statistically significant.

These results suggest that with limited data and small images, the convolutional bias of the ResNet tokenizer helps a transformer more than windowed attention or a hierarchical design alone. Parameter count, sequence length, and attention structure did not explain the differences between the models. Because the hybrid also adds depth and each model was trained with only one seed, a tokenizer ablation repeated over several seeds would be the next step to confirm this.

References

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[2] L. Beyer, X. Zhai, and A. Kolesnikov. Better plain ViT baselines for ImageNet-1k. *arXiv:2205.01580*, 2022.

[3] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[4] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. Swin Transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.